

Agri-Tech

AI-Powered QA

Software Engineering

Smart Crop Recommendation & Waste-to-Wealth System

AI-Powered QA Automation, Documentation & Software Engineering Assessment

Flask

HTML/CSS/JS

pytest

Playwright

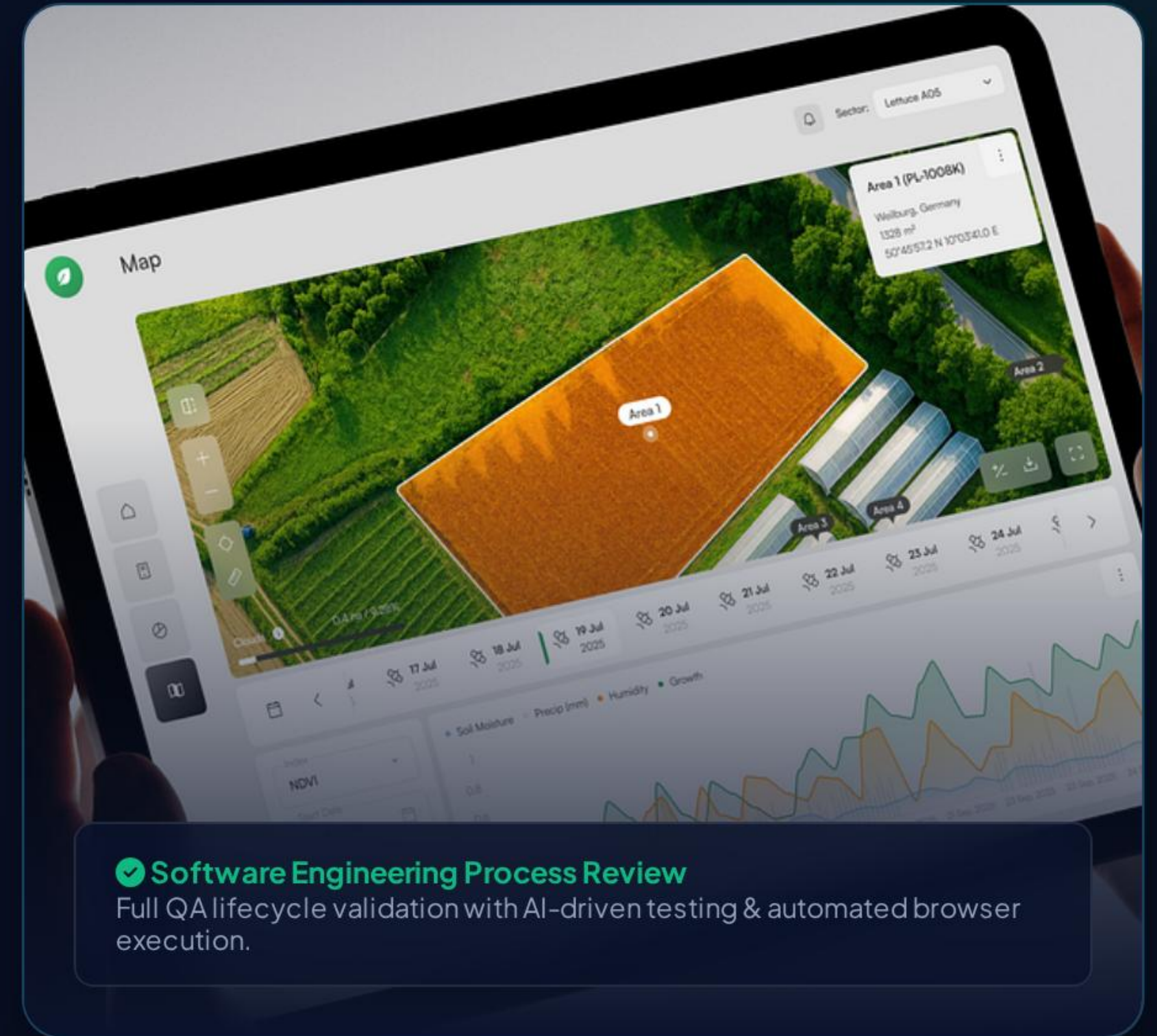
Chromium

AI-Assisted Dev

[STUDENT NAME]

[COLLEGE / INSTITUTION]

[DATE]



Software Engineering Process Review

Full QA lifecycle validation with AI-driven testing & automated browser execution.



Speaker Note (35s): "Good day. Today I am presenting my software engineering assessment project on a Smart Crop Recommendation & Waste-to-Wealth System. Beyond agricultural prediction, the core purpose of this assessment is to demonstrate a rigorous, AI-assisted software engineering and QA automation process using Python, Flask, pytest, and Playwright."

Project Overview & Agricultural Domain

SYSTEM CONTEXT

7 Environmental Input Parameters

The core recommendation engine relies on seven essential soil and weather inputs:

Nitrogen (N)

Phosphorus (P)

Potassium (K)

Temperature (°C)

Humidity (%)

Soil pH Level

Annual Rainfall (mm)

Target Crops Supported:

Rice, Wheat, Sugarcane, Maize, and regional cash crops.

Key Core Features

- **Crop Recommendation:** Data-driven crop selection logic.
- **Waste-to-Wealth Mapping:** Value-addition paths for agricultural waste.
- **Prediction History:** Chronological session history storage.
- **Reset Functionality:** Clears input forms and application state.
- **Input Validation:** Guardrails against missing or invalid parameters.

Assessment Context Note

While this application delivers actionable agricultural insights, the primary focus of this assessment is evaluating the **Software Engineering Process**: AI assistance, test-driven validation, regression detection, and automated browser testing.



Speaker Note (35s): "The application takes seven key environmental parameters like Nitrogen, Soil pH, and Rainfall to output tailored crop recommendations and circular waste-to-wealth reuse maps. However, our presentation today will quickly transition into how this web app was engineered, tested, and verified using modern AI-driven QA."

System Architecture & Testing Layer

ARCHITECTURE

PRODUCTION APPLICATION STACK FLOW



» Output Modules:

- Prediction History
- Waste Mapping
- Circular Wheel

AUTOMATED QA & TESTING INFRASTRUCTURE



Speaker Note (40s): "Here is our end-to-end system architecture. On the upper track, the user interacts with an HTML/CSS/JS frontend, which validates inputs before passing data to Flask. On the bottom track, pytest drives Playwright to programmatically spin up Chromium and run end-to-end tests against the live app."

Application Features & User Interface

SYSTEM FEATURES

Crop Recommendation

Processes environmental parameters and predicts the ideal crop.

Input Validation

Rejects missing, negative, and invalid numeric values instantly.

Prediction History

Stores valid past recommendations chronologically.

Reset Functionality

Clears forms, outputs, active wheel states, and history cleanly.

Waste-to-Wealth

Suggests monetization options for crop residues.

Circular Wheel

Visualizes Crop → Waste → Wealth → Next Crop loop.



[INSERT ACTUAL APPLICATION SCREENSHOT]

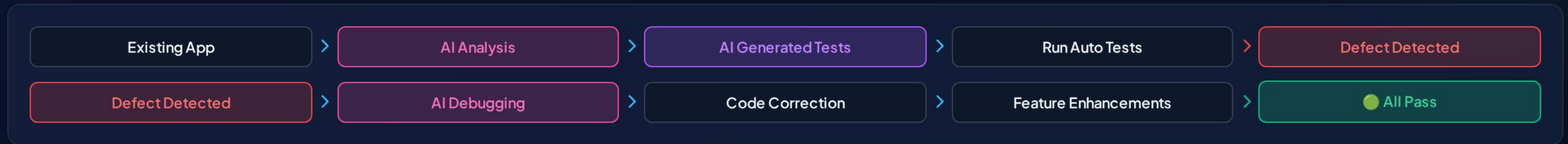
Main application dashboard view showing input parameter form, prediction card, and waste-to-wealth output panel.



Speaker Note (35s): "The application consists of 6 primary feature modules: prediction logic, input validation, prediction history, state reset, waste-to-wealth mapping, and circular visualization. On the right, you can see the application UI where these components come together."

🔧 AI-Assisted Engineering Workflow

AI + SOFTWARE ENGINEERING



☰ AI Applications in the Loop

- **Test Generation:** Synthesized edge-case and boundary Playwright scenarios.
- **Edge-Case Identification:** Suggested negative soil N-P-K and empty string checks.
- **Debugging Assistance:** Analyzed pytest exception traces to pin-point validation gaps.
- **Feature Implementation:** Assisted with history array state and UI reset handlers.
- **Documentation & Logging:** Structured architecture docs and AI log records.

👤 Human Verification Principle

"All AI suggestions were rigorously verified by executing the actual application and automated test suite. No AI code or test artifact was accepted without deterministic terminal execution proof."



Speaker Note (40s): "AI was leveraged as a force multiplier across the entire engineering lifecycle—from writing initial test cases to diagnosing failures. Crucially, as highlighted on the right, AI was treated as an assistant; every suggestion was verified through deterministic automated test execution."

✓ POSITIVE TESTS

- Rice recommendation
- Wheat recommendation
- Sugarcane recommendation
- Maize recommendation

📈 EDGE / BOUNDARY

- Extreme pH values (0, 14)
- High/low rainfall thresholds
- Boundary numeric checks
- Threshold behavior

⚠️ VALIDATION TESTS

- Required field checks
- Negative input rejection
- Non-numeric string inputs
- Toast error display

⚙️ FEATURE TESTS

- Prediction history storage
- Chronological ordering
- Reset state clearing
- UI element removal

 pytest

+

 Playwright

+

 Chromium

 "Tests interact with the application through a real browser, simulating user behaviour."



Deliberate Failure — Test Catches Defect

RED RUN

Deliberate Regression Injected

To verify the sensitivity and validity of the automated test suite, validation-related logic for negative values was intentionally altered.

Expected Behavior:

"Invalid Input" Toast Message

Actual Behavior:

Form accepted negative value / incorrect response

Automation Result:

 TEST FAILED

“ This demonstrated that the automated test suite was capable of detecting a regression and protecting against silent validation failures.



[INSERT ACTUAL FAILED PYTEST SCREENSHOT]

Actual terminal output demonstrating Pytest catching the deliberate assertion failure on negative inputs.



Speaker Note (40s): "A key requirement of software engineering assessment is proving that tests actually catch bugs. We performed a 'Red Run' by breaking negative value validation. As expected, Playwright failed the assertion, providing immediate feedback that our test was sensitive to real defects."

From Failure to Fix

BUG REMEDIATION

● Failed Test Detected



🔍 Analyze Failure



Identify Validation Gap



Correct Logic



● Run Test → PASS

Correction Details

The failure analysis indicated that negative agricultural input values were bypassing frontend guardrails and reaching prediction calculations.

The validation logic was updated to strictly inspect all 7 input values. Any value < 0 now immediately halts execution and displays the exact expected message: **"Invalid Input"**.

[FAILED TEST SCREENSHOT]

Terminal exception trace highlighting validation mismatch



[FIXED APPLICATION SCREENSHOT]

Corrected UI showing "Invalid Input" toast alert when negative parameter is submitted.



Speaker Note (35s): "Once the defect was caught, we analyzed the root cause, updated the validation logic in JavaScript/Flask to reject negative inputs, and re-executed Pytest. The test passed cleanly, demonstrating a complete red-to-green bug remediation loop."

Stage 3 — Feature Enhancement

FEATURE ITERATION

1. Prediction History Module

- Stores successful predictions (e.g., Rice, Sugarcane, Maize).
- Displays items in reverse chronological order.
- **Rule:** Invalid form submissions MUST NOT be appended to history.

2. Complete State Reset Module

- Clears all 7 input fields to blank/default states.
- Clears current recommendation and waste text outputs.
- Clears accumulated prediction history items.
- Removes active wheel states & animated resources.

Feature Request > AI Implementation > Automated Testing > **Final Verification**



[INSERT ACTUAL PREDICTION HISTORY SCREENSHOT]

Application UI demonstrating populated prediction history
log (Rice → Sugarcane → Maize) and functional Reset button.



Speaker Note (35s): "In Stage 3, we implemented feature enhancements: a chronological prediction history and a complete state reset button. AI helped generate the state management logic, and automated tests were added to verify that reset wipes all state without breaking existing features."



Final Test Results & Verification

VERIFICATION



ALL TESTS PASSED

`pytest -v` | 8 passed in 9.42s

Executed in headless Chromium environment via Playwright

✔ Verified Test Coverage

- ✔ Crop recommendations
- ✔ Required input fields
- ✔ Prediction history order
- ✔ Boundary behaviour
- ✔ Negative value rejection
- ✔ Reset clearing state



[INSERT ACTUAL FINAL PYTEST SCREENSHOT]

Terminal execution output confirming '8 passed in 9.42s'
without any remaining failures or warnings.



Speaker Note (35s): "This is our final verification slide. The complete test suite consisting of 8 comprehensive E2E test cases executed cleanly in 9.42 seconds. This confirms that all feature requirements, validation rules, and edge cases are 100% satisfied."

project-root/

- README.md
- app.py (Flask Application)
- test_app.py (Playwright Test Suite)
- docs/
 - ARCHITECTURE.md
 - DESIGN.md
 - USER_GUIDE.md
- evidence/
 - baseline-test.txt
 - red-run.txt
 - finaltest.txt
 - AI_CHANGE_LOG.md
- static/ & templates/

Assessment Evidence Package

- **System Documentation:** ARCHITECTURE.md and DESIGN.md detailing data flows.
- **Audit Trail:** AI_CHANGE_LOG.md capturing AI prompting & human verification.
- **Test Log Proofs:** Raw terminal outputs saved in evidence directory.
- **Video & Slides:** Video walk-through and presentation deck.

i "The documentation makes the project completely reproducible, understandable, and reviewable by external software engineering auditors."





Engineering Learnings & Conclusion

SUMMARY

💡 Five Key Takeaways

- 1 **AI Velocity:** AI significantly accelerates test suite generation and boilerplate setup.
- 2 **Objective Feedback:** Automated browser tests eliminate manual verification bias.
- 3 **Deliberate Red Runs:** Injecting artificial defects validates that assertions are truly working.
- 4 **Regression Guardrails:** New features must be continuously verified against existing test suites.
- 5 **Human Engineering Judgement:** AI is an assistant; final validation requires human execution proof.

"AI was used as an engineering assistant, while actual application execution and automated test results were used to verify the implementation."

THANK YOU



Speaker Note (45s): "In conclusion, this project demonstrated that while AI can dramatically accelerate software and QA workflows, human engineering judgement and automated test suites remain essential to ensure quality and correctness. Thank you for your time, and I welcome any questions."

Image Sources



<https://cdn.dribbble.com/userupload/44270652/file/70cac45b2fae62e14a68931b8ee0e9c5.png?resize=752x&vertical=center>

Source: dribbble.com